

PRX | Sprint 1

Enlaces web y tareas en notas. Documento actualizado con el código vigente.

Proyecto	PRX: repositorios, notas y colaboración
Equipo	Guarachi Ramos Vania; Huamani Huaypuna Jose Andres; Marupa Arnez Fernando; Ramirez Arancibia Camila Fatima.
Base documental	Presentacion_Sprint_1_PRX (1).docx y código del proyecto. Revisión: 09/10/2026.
Alcance	Agregar y administrar enlaces HTTP/HTTPS y tareas en las notas, con persistencia y permisos.

Objetivo y resultado

Convertir la nota en un recurso de consulta y seguimiento. El usuario puede guardar enlaces, abrir recursos externos, agregar o eliminar tareas y marcar su estado. Los datos se serializan en FormData y se guardan como JSON mediante Prisma y MySQL.

Funciones implementadas

Enlaces: agregar varias URL, completar HTTPS cuando falta, validar HTTP/HTTPS, eliminar y consultar enlaces desde Ver nota.

Tareas: agregar y eliminar durante la edición, marcar o desmarcar desde el detalle y conservar el estado al consultar nuevamente.

Permisos: autenticación y acceso según propietario, membresía y visibilidad del repositorio.

Recorrido de demostración

1. Abrir Mis repositorios y entrar en Notas.
2. Abrir una nota y pulsar Editar nota.
3. Agregar una URL y una tarea; guardar.
4. Abrir el enlace y marcar la tarea desde Ver nota.
5. Recargar y comprobar que los datos permanecen.

Relación con Sprint 2

Sprint 1 incorpora enlaces y tareas. Sprint 2 amplía multimedia y edición de imágenes sin retirar esas listas. El handler vigente utiliza `updateWithFiles` para conservar los adjuntos al cambiar contenido, tareas o enlaces.

1. Frontend: enlaces y tareas

Fragmentos reales del código actual. La numeración corresponde a esta revisión.

Contrato compartido de la nota

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-frontend/src/app/features/notes/domain/models/note.model.ts

Fragmento real, líneas 1-16. Los modelos describen las listas que utilizan la interfaz y las respuestas HTTP.

```
1 import { NoteFileModel } from '@features/notes/domain/models/note-file.model';
2 import { CurrentUserModel } from '@shared/models/current-user.model';
3
4 export interface NoteTaskModel {
5   id: string;
6   title: string;
7   completed: boolean;
8 }
9
10 export interface NoteLinkModel {
11   id: string;
12   url: string;
13 }
14
15 export interface NoteModel {
16   id: number;
```

Actualizar el estado de una tarea

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-frontend/src/app/features/notes/presentation/pages/note-detail-page/note-detail-page.component.ts

Fragmento real, líneas 217-221. La señal cambia el estado; fuera de edición se invoca persist para guardar sin abrir el formulario.

```
217 protected toggleTask(task: NoteTaskModel): void {
218   if (!this.canEdit() || this.saving()) return;
219   this.tasks.update((tasks) => tasks.map((item) => item.id === task.id ? { ...item, completed:
    !item.completed } : item));
220   if (!this.editing()) this.persist(false);
221 }
```

Normalizar y validar URL

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-frontend/src/app/features/notes/presentation/pages/note-detail-page/note-detail-page.component.ts

Fragmento real, líneas 465-479. Completa HTTPS y limita los protocolos. El backend vuelve a validar la entrada.

```
465 private normalizeUrl(value: string): string {
466   const trimmed = value.trim();
467   if (!trimmed) return '';
468   return /^https?:\/\//i.test(trimmed) ? trimmed : `https://${trimmed}`;
469 }
470
471 private isValidUrl(value: string): boolean {
472   try {
473     const url = new URL(value);
474     return url.protocol === 'http:' || url.protocol === 'https:;
475   } catch {
476     return false;
477   }
478 }
479 }
```

2. Integración con el backend

Pantalla / fachada / API multipart / controlador / handler / Prisma / MySQL.

Enviar tareas, enlaces y adjuntos conservados

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-frontend/src/app/features/notes/infrastructure/api/note.api.ts

Fragmento real, líneas 70-77. JSON mantiene las listas dentro del mismo FormData que los archivos.

```
70 const formData = new FormData();
71 formData.append('title', data.title);
72 formData.append('content', data.content);
73 formData.append('tasks', JSON.stringify(data.tasks));
74 formData.append('links', JSON.stringify(data.links));
75 formData.append('retainedFileIds', data.retainedFileIds.join(','));
76 files.forEach((file) => formData.append('files', file, file.name));
```

Validación del caso de uso

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-backend/src/modules/notes/application/commands/update-note/update-note.handler.ts

Fragmento real, líneas 49-55. Los datos se analizan antes de actualizar. Las validaciones completas están en parseTasks y parseLinks al final del archivo.

```
49 );
50
51 if (duplicate) {
52   throw new BadRequestException(NOTE_MESSAGES.DUPLICATE_TITLE);
53 }
54 const retainedIds = command.dto.retainedFileIds === undefined
55   ? note.files.map((file) => file.id!)
```

Escritura de contenido estructurado

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-backend/src/modules/notes/infrastructure/persistence/prisma-note.repository.ts

Fragmento real, líneas 190-202. La transacción actualiza la nota y sus listas. Esta operación mantiene la compatibilidad con el multimedia de Sprint 2.

```
190   data: { status: 0, updatedBy: data.updatedBy },
191 });
192 const note = await tx.note.update({
193   where: { id, status: 1 },
194   data: {
195     title: data.title,
196     content: data.content,
197     tasks: data.tasks as unknown as Prisma.InputJsonValue,
198     links: data.links as unknown as Prisma.InputJsonValue,
199     updatedBy: data.updatedBy,
200     noteFiles: { create: files },
201   },
202   include: { noteFiles: { where: { status: 1 } }, createdByUser: true },
```

Dónde revisar la validación

UpdateNoteHandler limita las tareas a 100, cada título a 200 caracteres, los enlaces a 50 y cada URL a 2048 caracteres. Rechaza listas inválidas y protocolos distintos de HTTP o HTTPS. Antes de escribir verifica la nota, el repositorio y los permisos del usuario.

3. Arquitectura y organización

Patrones y principios observados en PRX.

Patrón	Aplicación concreta
Facade	NoteFacade conecta la interfaz con NoteApi y el estado.
CQRS	NotesController delega comandos y consultas a buses NestJS.
Repository	NoteRepository abstrae persistencia; PrismaNoteRepository realiza las operaciones.
Mapper	Mapeadores convierten datos Prisma en entidades y respuestas; listas ausentes se presentan como arreglos vacíos.
Inyección de dependencias	NotesModule vincula contratos con implementaciones.

SOLID: aplicación parcial

Las capas separan presentación, aplicación y persistencia. Los handlers dependen del contrato NoteRepository. El detalle de nota todavía concentra tareas, enlaces, archivos y edición; las dependencias concretas de API y almacenamiento son oportunidades de mejora. No se afirma cumplimiento absoluto de los cinco principios.

Kanban y XP en el proyecto

Las actividades realizadas pueden organizarse en validación de enlaces, administración de tareas, integración con datos y revisión. Los cambios pequeños, las pruebas y la refactorización apoyan XP. No se verificó un tablero formal, programación en parejas ni TDD estricto.

Tipos de integración

Incremental: conectar y comprobar listas, persistencia y pantalla por pasos.

Top-Down: empezar en NoteDetail y simular NoteApi.

Bottom-Up: comprobar repositorios y handlers antes de conectar la pantalla.

Big Bang: integrar todo al final; es una alternativa explicativa, no una práctica confirmada del equipo.

4. Validación y mapa del código

Resultados ejecutados, límites y rutas para ubicar la implementación.

Comprobación	Resultado al 09/10/2026
Compilación	Frontend y backend compilaron correctamente. Angular muestra tres avisos de presupuesto SCSS, sin error de compilación.
Pruebas unitarias	21 pruebas backend y 22 frontend aprobadas. Incluyen archivos, fallos de guardado, registro y utilidades de imagen; no constituyen cobertura total del proyecto.
Integración real	Actualización de una nota con tarea completada y enlace: datos recuperados correctamente desde MySQL. Adjuntos conservados y permisos comprobados con API HTTP.
Límites	Las tareas se administran desde detalle/edición. La creación inicial no ofrece tareas. No se certificó cada navegador, rol o módulo de producción.

Rutas frontend desde prx-frontend/

src/app/features/notes/presentation/pages/note-detail-page/: componente, HTML, SCSS y pruebas.

src/app/features/notes/infrastructure/api/note.api.ts: envío FormData.

src/app/features/notes/application/facades/note.facade.ts: coordinación.

src/app/features/notes/domain/models/note.model.ts: contratos.

src/app/features/notes/infrastructure/store/note.store.ts: estado.

Rutas backend desde prx-backend/

src/modules/notes/presentation/controllers/notes.controller.ts: endpoints.

src/modules/notes/application/commands/update-note/update-note.handler.ts: validación y permisos.

src/modules/notes/domain/repositories/note.repository.ts: contrato.

src/modules/notes/infrastructure/persistence/prisma-note.repository.ts: transacciones.

prisma/schema.prisma: modelo Note, campos tasks y links.

API y documentación

PUT /prx/notes/:id guarda cambios; GET /prx/notes/:id recupera la nota. El Manual Técnico original permanece intacto localmente y fuera de GitHub por contener contraseñas. Sprint_1_PRX.pdf y

Proyecto_PRX_Informe_Unificado_Sprint_2.pdf documentan el estado actualizado sin publicar credenciales.